Equity Derivatives & ML

Option Pricing with Black–Scholes, Monte Carlo and Neural Networks

Bilal Ashraf — MSc Mathematical Engineering, Université Côte d'Azur

1. Question

Monte Carlo can price almost any derivative, but it is slow: each price requires many simulated paths, and a trading desk reprices thousands of options many times a day. A machine learning model can be trained once, offline, and then return prices almost instantly.

Can a neural network reproduce the price of an option faster than Monte Carlo, and with what error?

The test is run on a European call, because the Black–Scholes formula gives the exact price and makes the error measurable.

2. Definitions

Equity derivative. Contract whose value depends on a stock or an index.

European call. The right to buy the stock at strike K at maturity T; payoff $\max(S_T - K, 0)$.

Moneyness. The ratio S/K between the stock price and the strike.

Black–Scholes. Closed-form price of a European option under constant volatility σ and rate r.

Monte Carlo. Price estimated as the average discounted payoff over N simulated scenarios. Its error decreases as $1/\sqrt{N}$.

Neural network (MLP). A function made of layers of weighted sums and non-linear activations, whose weights are fitted on examples.

3. Method

Black–Scholes (reference). With $d_1 = \dfrac{\ln(S/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}$ and $d_2 = d_1 - \sigma\sqrt{T}$,

$$C = S\,\Phi(d_1) - K\,e^{-rT}\,\Phi(d_2).$$

Monte Carlo. Under the risk-neutral measure the stock follows a geometric Brownian motion, so

$$S_T = S_0 \exp\Big((r - \tfrac{\sigma^2}{2})T + \sigma\sqrt{T}\,Z\Big), \qquad \hat{C} = \frac{e^{-rT}}{N}\sum_{i=1}^{N} \max\big(S_T^{(i)} - K, 0\big).$$

Neural network.

- *Data.* 100,000 options drawn uniformly: moneyness in $[0.6, 1.4]$, maturity in $[0.1, 2]$ years, volatility in $[10\%, 60\%]$, rate in $[0\%, 6\%]$; 20,000 others are kept as a test set.
- *Target.* The price is homogeneous in (S, K), so the network learns C/K as a function of S/K. This removes one input and any scale problem.
- *Model.* Two hidden layers of 64 neurons, ReLU activation, standardised inputs, early stopping on a validation set (`scikit-learn`).

4. Code

Full code: `ml_option_pricing.py` (130 lines, NumPy, SciPy, scikit-learn and Matplotlib).

```
def sample_options(n, rng):
    x = np.column_stack([
        rng.uniform(0.6, 1.4, n),      # moneyness S / K
        rng.uniform(0.1, 2.0, n),      # maturity in years
        rng.uniform(0.10, 0.60, n),    # volatility
        rng.uniform(0.00, 0.06, n),    # risk-free rate
    ])
    y = black_scholes_call(x[:, 0], 1.0, x[:, 1], x[:, 2], x[:, 3])   # price / K
    return x, y

x_train, y_train = sample_options(100_000, rng)
scaler = StandardScaler().fit(x_train)
model = MLPRegressor(hidden_layer_sizes=(64, 64), activation="relu",
                     early_stopping=True, random_state=42)
model.fit(scaler.transform(x_train), y_train)
```

5. Results

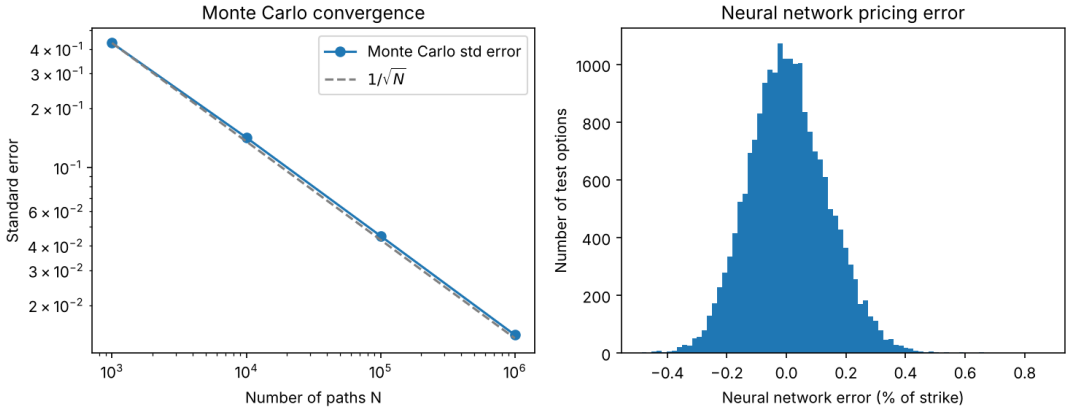
Monte Carlo convergence. Reference option: $S = K = 100$, $T = 1$ year, $\sigma = 20\%$, $r = 3\%$. The Black–Scholes price is 9.4134.

Paths N	Monte Carlo price	Standard error	Absolute error
1,000	8.8811	0.4293	0.5323
10,000	9.3590	0.1421	0.0544
100,000	9.3936	0.0449	0.0198
1,000,000	9.4200	0.0141	0.0066

The standard error is divided by about 3.2 ($\sqrt{10}$) each time N is multiplied by 10: one more digit of precision costs 100 times more paths.

Neural network accuracy (20,000 test options never seen in training).

Metric	Value
R^2	0.99992
Mean absolute error	0.106% of strike
RMSE	0.134% of strike
99th percentile of the absolute error	0.351% of strike
Mean relative error (options worth more than 1% of strike, 89% of the test set)	1.31%



Speed. Time to price the 20,000 test options, on a 2-core machine:

Method	Time
Black–Scholes (closed form)	3 ms
Neural network	26 ms
Monte Carlo, 100,000 paths per option (extrapolated from 200 options)	57 s

The network is about 2,000 times faster than Monte Carlo, after a one-off training of 12.5 seconds. Timings depend on the machine; the order of magnitude is what matters.

Reading.

- The network prices with an average error of 0.11% of the strike, about 11 cents on a 100 strike. A Monte Carlo with 100,000 paths has a standard error of 4.5 cents on the reference option: the network is less precise, but thousands of times faster.
- The relative error (1.3%) is higher than the absolute one because cheap, out-of-the-money options have small prices. This is where the network is weakest.
- For a European call the network is useless in practice, since Black–Scholes is exact and faster. The result is a proof of concept: the same approach applies to products with no closed form, where the training prices come from Monte Carlo.

6. Limitations and extensions

- Training labels are exact Black–Scholes prices. With Monte Carlo labels, the network would also have to average out simulation noise.
- The network is only valid inside the parameter ranges it was trained on and gives no error bound.
- Nothing guarantees absence of arbitrage (monotonicity in S, convexity in K), and the Greeks obtained by differentiating the network are noisier than the prices.
- Extensions: train on an exotic product (Asian or barrier option) priced by Monte Carlo, or on the Heston model, where the speed gain is real.